

Team Implementation Report

Feature: Email Subscription Service with Double Opt-In

Repository: Azmith47/public-safety-alerts-aggregator

Goal: Add a secure, reliable email alert subscription system for public safety notifications.

1. Why We Are Implementing This

We need users to subscribe to safety alerts by email in a way that is:

- **Easy to use**
- **Accurate** (real email owners only)
- **Secure**
- **Compliant** with anti-spam and privacy expectations

To achieve this, we will use **Double Opt-In**.

2. What Double Opt-In Means

A user is subscribed only after two steps:

1. User submits email + optional preferences (region/category/frequency).
2. User clicks a confirmation link sent to their email.

Until confirmation, status remains **Pending**.

After confirmation, status becomes **Confirmed**.

3. Scope of This Feature

In Scope

- Subscription form in UI
- Confirmation email with secure tokenized link
- Confirmation endpoint
- Unsubscribe endpoint/link
- Preference management (region/category/frequency)
- Delivery only to confirmed users
- Basic logs/metrics

Out of Scope (Initially)

- Advanced campaign analytics
 - Complex segmentation rules
 - Multi-channel notifications (SMS/push)
-

4. Proposed User Flow

Subscribe

- User enters email and preferences.
- Backend stores record as `pending`.
- Confirmation email is sent.
- UI shows generic message: "Check your email to confirm."

Confirm

- User clicks link with token.
- Backend verifies token and expiry.
- Status changes `pending` → `confirmed`.
- User sees success page.

Unsubscribe

- Every alert email includes unsubscribe link.
- Clicking link sets `status = unsubscribed`.
- User sees confirmation page.

Alert Delivery

- Jobs only target users with `status = confirmed`.
 - Filters apply based on stored preferences.
-

5. Data Model (Shared Understanding)

Suggested `subscriptions` Table Fields

Field	Description
<code>id</code>	UUID (primary key)
<code>email</code>	Normalized lowercase, unique
<code>status</code>	<code>pending</code> , <code>confirmed</code> , <code>unsubscribed</code> , <code>bounced</code>

Field	Description
preferences_json	Regions/categories/frequency
confirm_token_hash	Hashed confirmation token
confirm_token_expires_at	Token expiry timestamp
unsubscribe_token_hash	Hashed unsubscribe token
created_at	Record creation timestamp
updated_at	Last update timestamp
confirmed_at	Confirmation timestamp
last_email_sent_at	Last email delivery timestamp
bounce_count	Number of bounces

Indexes

- email (unique)
- status
- (status, updated_at)

6. Security Recommendations (Important)

Token Security

- Use cryptographically secure random tokens.
- Store only **hashed tokens** in DB (never plaintext).
- Set confirmation token expiry (24–48 hours).
- Invalidate token immediately after successful use.

Endpoint Protection

- Add rate limits on:
 - Subscribe
 - Resend confirmation
 - Unsubscribe
- Optional bot defense: honeypot/reCAPTCHA.
- Use generic success responses to prevent email enumeration.

Input and Data Protection

- Validate and sanitize all inputs.
- Normalize email format before processing.
- Encrypt data in transit (HTTPS only).
- Restrict DB and SMTP credentials via environment secrets.

Mail and Domain Security

- Configure SPF, DKIM, and DMARC for sender domain.
- Use a trusted provider (SES/SendGrid/Postmark/Mailgun).
- Monitor bounce/complaint webhooks and suppress bad addresses.

Application Security Operations

- Add structured audit logs for:
 - Subscription requested
 - Confirmed
 - Unsubscribed
 - Email bounced/complaint
 - Avoid storing unnecessary personal data.
 - Define retention policy for inactive/unsubscribed records.
-

7. Compliance and Trust Requirements

- Clear consent language next to subscribe button.
 - Link privacy policy and terms.
 - Include unsubscribe in every non-transactional alert email.
 - Store evidence of consent (timestamp/source).
 - Keep communication relevant to public safety purpose.
-

8. API Plan (Draft)

Endpoint	Method	Description
/api/subscriptions	POST	Creates or refreshes pending state and sends confirmation
/api/subscriptions/confirm	GET	Confirms pending subscription (query param: token)
/api/subscriptions/resend-confirmation	POST	Re-sends confirmation email
/api/subscriptions/unsubscribe	POST	Unsubscribes user via secure token
/api/subscriptions/preferences	POST	Updates preferences for authenticated token holder

9. Team Implementation Plan by Phase

Phase 1 — Setup

Owner: Backend + DevOps

- Choose email provider
- Add environment variables/secrets management
- Create database migration and model
- Build token/hash utility

Phase 2 — Core Feature

Owner: Backend + Frontend

- Build subscribe/confirm/unsubscribe endpoints
- Add subscribe and confirmation UI
- Add confirmation email template

Phase 3 — Delivery Integration

Owner: Backend

- Build scheduled sender (digest or instant)
- Filter by `confirmed` + preferences
- Add unsubscribe link handling

Phase 4 — Security Hardening

Owner: Security + Backend

- Rate limits, abuse controls
- Bounce/complaint webhook processing
- Audit logs + monitoring alerts

Phase 5 — QA and Release

Owner: QA + Product

- End-to-end tests
- Template rendering checks (desktop/mobile)
- Staged rollout and monitoring

10. QA Checklist

- New subscription creates pending record
- Confirmation email is sent
- Confirmation link activates account once
- Expired/invalid token handled safely
- Only confirmed users receive alerts
- Unsubscribe works from email link

- Rate limiting works under abuse attempts
 - No user enumeration via API responses
 - Logs capture critical events
-

11. Risks and Mitigations

Risk	Mitigation
Spam folder delivery	SPF/DKIM/DMARC + provider warm-up
Automated abuse	Rate limits + bot checks
Token leakage/replay	Hashed tokens + short expiry + one-time use
Compliance gaps	Consent text + policy links + unsubscribe hygiene

12. MVP Recommendation (First Release)

Launch with:

- Double opt-in flow
- Region/category preferences
- Daily digest alerts
- One-click unsubscribe
- Basic operational dashboard (pending/confirmed/unsubscribed/bounced)

Then iterate with instant critical alerts and richer preferences.